Figure 6.15 Possible cases comparing a search string to a node's links.
(1) An edge’s label completely match part of the string.
(2) An edge’s label and the search string have a common prefix that is shorter than both strings.
(3) The search string is a prefix of one of the edge’s label.
(4) The search string has no common prefix with any of the edges.

The alternative solutions to implement this dictionary for edges are the usual: balanced search trees, which would guarantee logarithm search *and* insertion, or hash tables. The latter is illustrated in figure 6.14: we can keep a dictionary whose keys are characters and whose values are the full string labels of the node’s edges, together with a reference to the children linked by the edge. This solution requires $0(k)$ additional space for a node with k children, and worst-case it will require $0(|\sum|)$ extra space per node.

Despite requiring more space and a little book keeping on insertion and deletion to update the hash table, this solution allows amortized constant-time lookup when searching the path for a key.

Independently of the implementation, the first step will be comparing the first character of the input string to the first character of the edges’ label.

Overall, we have 4 possible cases, illustrated in figure 6.15: